Visvesvaraya Technological University

Jnana Sangama, Belagavi – 590014

A PROJECT WORK PHASE-2 REPORT ON

Bluetooth Low Energy and Surveillance Camera-based Navigation System for Visually Impaired People

Submitted in fulfillment of the requirements for the award of the degree of
Bachelor of Engineering in Computer Science and Engineering

SUBMITTED BY

Ms. Aishwarya Kumar Telasinge	2AG23CS007
Ms. Anuja Sutar	2AG23CS016
Ms. Aishwarya Channashetti	2AG23CS006
Ms. Chandrika Bannimatti	2AG23CS032

UNDER THE GUIDANCE OF

Prof. Avanti Patil
Assistant Professor, Dept. of CSE, AITM, Belagavi

Angadi Institute of Technology & Management

Department of Computer Science and Engineering, Belagavi – 590009

2025 – 2026

Bluetooth Low Energy and Surveillance Camera-based Navigation System for Visually Impaired People

Project Work Phase-2 Report · Visvesvaraya Technological University, Belagavi Department of Computer Science and Engineering, Angadi Institute of Technology and Management, Belagavi · 2025–2026

Submitted by	Ms. Aishwarya Kumar Telasinge (2AG23CS007) · Ms. Anuja Sutar (2AG23CS016) · Ms. Aishwarya Channashetti (2AG23CS006) · Ms. Chandrika Bannimatti (2AG23CS032)
Guide	Prof. Avanti Patil, Assistant Professor, Dept. of CSE, AITM Belagavi
Implementation status	2,167 lines of Python / C++ across 15 source files · 62 unit tests passing · 11/11 environment smoke tests passing

CHAPTER 1

Introduction

1.1 Overview

Independent mobility is the single capability that most strongly determines quality of life for a person with a visual impairment. Outdoors, the problem is partially solved: GPS combined with a smartphone gives usable turn-by-turn guidance at a resolution of a few metres. Indoors, that entire stack collapses. Satellite signals are attenuated by roofs and walls, and even where a fix is obtained its error is larger than the corridors a user must walk through. Campuses, hospitals, railway stations and shopping complexes — exactly the buildings where an unfamiliar route is most likely and most consequential — are therefore the least supported environments.

The traditional aids fill part of this gap and no more. A white cane senses only what is within one arm's length and only near the floor; tactile paving encodes a fixed route that cannot warn about the chair that was moved into it this morning. Neither can answer the two questions that matter while walking: *where am I on the route right now*, and *what is in front of me that I do not know about*.

This project answers both by using infrastructure that the building already has. Surveillance cameras are already mounted in most public indoor spaces, and Bluetooth Low Energy radios are already in every smartphone. The system pairs the two: **the camera performs localisation and obstacle detection from a third-person overhead viewpoint, while BLE answers the identity question — which of the people visible in the frame is the registered user.** The output is spoken guidance delivered in short, imperative phrases: *"Stop. Obstacle ahead."*, *"Caution. Stairs ahead."*, *"You are drifting left. Move right."*

1.2 Why a third-person viewpoint

Almost every published assistive-navigation system is *ego-centric*: the sensor is worn by the user, so it sees what the user faces. That geometry has a structural weakness — an obstacle outside the sensor's field of view, or occluded by the user's own body, is invisible until it is too late. A ceiling-mounted surveillance camera looking down at 25–35° sees the user, the floor plane, and everything on that floor plane *simultaneously*, including obstacles behind and beside the user. Blind spots in the ego-centric formulation are ordinary pixels in this one.

The trade-off is that the system becomes infrastructure-bound: it works inside the calibrated camera's coverage and nowhere else. For the target deployment — a known building with a known set of safe routes — that is an acceptable and deliberate constraint.

1.3 Scope of this report

Phase-1 established the problem, surveyed the literature, and proposed the architecture. This Phase-2 report documents the **realised system**: the requirement specification it was built to, the architecture and UML models that describe it, and a working implementation of every core subsystem — capture, detection, tracking, floor-plane calibration, BLE presence, user matching, navigation policy and speech output — verified by an automated test suite that runs without any hardware attached.

1.4 A deliberate deviation from the Phase-1 proposal

Phase-1 proposed BLE *sniffers* distributed through the building, with RSSI fingerprints fed to DNN/CNN/LSTM models to estimate position, and Human Body Orientation Estimation (HBOE) to recover the user's facing direction. Under the project exemption granted for Phase-2, no BLE sniffer array is deployed. Two consequences follow, and both were turned into design decisions rather than worked around:

PHASE-1 PROPOSAL	PHASE-2 REALISATION	JUSTIFICATION
BLE trilateration from a sniffer array	BLE reduced to **presence + coarse range** from a single adapter	With one receiver, trilateration is mathematically impossible. RSSI is retained where it is genuinely reliable: *is the registered device in the room*, and a coarse distance used only as a tie-breaker.
DNN/CNN/LSTM RSSI localisation	**Homography-based visual localisation** on the floor plane	A four-point homography fitted to a tape-measured floor rectangle converts any foot pixel to metres with sub-decimetre error, with no training data and no inference cost. It is strictly more accurate than RSSI regression.
HBOE for user heading	**Heading from displacement** over a 0.8 s window	Body orientation only matters while walking, and while walking the direction of travel *is* the heading. This removes a second neural network from the critical path on a CPU-only machine.
YOLOv7	**YOLOv8n + ByteTrack**	v8n is smaller and faster on CPU, and Ultralytics ships ByteTrack integrated — persistent track IDs are required to follow one specific human across frames.

The resulting split is stated in one line: **vision does the localisation, BLE does the identification.**

CHAPTER 2

Problem Statement

Indoor navigation for visually impaired individuals remains unsolved because no single, affordable, deployable system simultaneously provides accurate indoor positioning, real-time obstacle awareness, route adherence and timely audio guidance.

GPS, the technology that solved outdoor navigation, is unusable indoors: satellite signals are weak or absent, and the resulting error exceeds the width of the corridors a user must walk. Traditional aids — the white cane and tactile paving — offer only near-field, floor-level, fixed-route assistance and cannot report position, cannot warn about dynamic obstacles, and cannot tell a user that they have drifted off the safe path.

The research systems that do address indoor assistance each solve one part of the problem in isolation. Wireless-localisation work produces position estimates but no obstacle awareness. Wearable obstacle-avoidance devices detect what is directly ahead but do not know where the user is in the building. Vision systems achieve high detection accuracy but assume a GPU that a deployable, low-cost installation will not have. Very few systems integrate localisation, obstacle detection, deviation correction and speech into a single real-time platform, and fewer still do so without requiring the user to purchase and wear additional hardware.

Six specific sub-problems follow from this, and each is addressed by a named subsystem of the implemented design:

#	SUB-PROBLEM	ADDRESSED BY
PS-1	No indoor position fix of usable accuracy	Four-point homography mapping camera pixels to floor metres (`world.FloorPlane`)
PS-2	Obstacles in navigation blind spots are invisible to ego-centric sensors	Overhead third-person camera view + YOLOv8n detection (`detector.Detector`)
PS-3	Multiple people in frame — which one is the registered user?	BLE presence + RSSI-agreement scoring with a stability streak (`matcher.TargetMatcher`)
PS-4	Silent drift off the safe route goes uncorrected	Perpendicular deviation from a stored polyline path (`world.NavPath`)
PS-5	Guidance that arrives late is worse than no guidance	Fixed 10 Hz decision loop decoupled from detector speed, with velocity extrapolation (`guidance.GuidanceLoop`)
PS-6	Extra wearable hardware is costly and uncomfortable	User carries only an ordinary BLE-advertising device; no wearable sensors required

CHAPTER 3

Objectives

ID	OBJECTIVE	REALISATION IN PHASE-2
O1	Develop an indoor navigation system that localises a user using surveillance cameras and BLE, without GPS.	Implemented. Homography floor mapping + BLE presence gate. Position reported in metres in a floor coordinate frame.
O2	Detect static and dynamic obstacles in the user's walking corridor in real time.	Implemented. YOLOv8n over 17 indoor COCO obstacle classes plus untargeted persons, with a 2.5 s persistence cache.
O3	Identify which detected person is the registered user, reliably, when several people are present.	Implemented. Sticky lock with a 5-frame stability requirement and a 2 s occlusion grace period.
O4	Detect deviation from a predefined safe path and issue correction guidance.	Implemented. Signed perpendicular distance to the nearest path segment; correction spoken when it exceeds 0.6 m.
O5	Deliver guidance as prioritised, non-overlapping speech, timely enough to act on.	Implemented. Five-level priority policy, single-slot pre-emptive speech queue, per-message cooldowns.
O6	Improve independence without requiring the user to wear additional hardware, and run on commodity low-cost hardware.	Implemented. CPU-only inference at 26–28 FPS on the development machine; the user carries only a BLE beacon/phone.
O7	Degrade gracefully rather than fail when a subsystem is unavailable.	Implemented. Every subsystem has a defined degradation path (Section 8.6).

CHAPTER 4

System Requirement Specification

4.1 Purpose

This SRS specifies the functional and non-functional requirements of the BLE + Surveillance Camera Navigation System. The system's users are (a) a visually impaired person carrying a BLE-advertising device, who consumes only the audio output, and (b) an operator/administrator who performs the one-time calibration and starts the system.

4.2 Functional requirements

ID	REQUIREMENT	PRIORITY	STATUS
FR-01	The system shall capture video continuously from a ceiling-mounted camera or a recorded video file.	High	Implemented
FR-02	The system shall always process the newest available frame and discard stale buffered frames from a live camera.	High	Implemented
FR-03	The system shall detect persons in each processed frame and assign each a persistent track identifier.	High	Implemented
FR-04	The system shall detect indoor obstacle classes (chair, couch, table, bench, bicycle, potted plant, bottle, backpack, suitcase and others) in each processed frame.	High	Implemented
FR-05	The system shall retain a detected obstacle on the internal map for a configurable time-to-live after it ceases to be detected.	Medium	Implemented
FR-06	The system shall provide an interactive calibration utility that computes a pixel-to-floor homography from four operator-clicked points of a measured floor rectangle.	High	Implemented
FR-07	The calibration utility shall allow the operator to trace and store a predefined safe path as a polyline, in both pixel and metre coordinates.	High	Implemented
FR-08	The system shall convert the bottom-centre pixel of any detection box into floor coordinates in metres.	High	Implemented
FR-09	The system shall scan for BLE advertisements and identify the registered beacon by MAC address, or by advertised name when no address is configured.	High	Implemented
FR-10	The system shall smooth RSSI over a moving window and convert it to a coarse distance using a log-distance path-loss model.	Medium	Implemented
FR-11	The system shall report the user as present only when a fresh advertisement has been seen within the loss timeout and the smoothed RSSI exceeds the presence threshold.	High	Implemented
FR-12	The system shall select, among all tracked persons, the one whose camera-measured distance best agrees with the BLE-estimated distance, and shall require agreement over consecutive frames before locking.	High	Implemented

ID	REQUIREMENT	PRIORITY	STATUS
FR-13	The system shall retain an existing target lock while that track survives, and for a grace period after brief occlusion.	High	Implemented
FR-14	The system shall estimate the user's speed and heading from position displacement over a configurable time window.	High	Implemented
FR-15	The system shall extrapolate the user's position between detector passes, bounded to a maximum extrapolation horizon.	High	Implemented
FR-16	The system shall express every obstacle in the user's own forward/lateral frame and discard those outside the walking corridor.	High	Implemented
FR-17	The system shall issue an immediate stop instruction when an obstacle lies within the stop distance in the corridor.	High	Implemented
FR-18	The system shall issue a stairs caution when stairs are detected within the stairs warning distance.	High	Policy implemented; detector pending fine-tuned weights
FR-19	The system shall issue an avoidance instruction naming the side with greater clearance when an obstacle lies within the warning distance.	High	Implemented
FR-20	The system shall issue a correction instruction when the user's perpendicular deviation from the path exceeds the deviation threshold.	High	Implemented
FR-21	The system shall issue a periodic all-clear instruction when no higher-priority condition holds.	Medium	Implemented
FR-22	The system shall emit exactly one instruction per decision cycle — the highest-priority one.	High	Implemented
FR-23	The system shall suppress a repeated identical instruction within its configured cooldown.	Medium	Implemented
FR-24	A pending spoken instruction shall be replaced by a newer instruction of equal or higher urgency rather than queued behind it.	High	Implemented
FR-25	The system shall render an on-screen view showing detection boxes, the locked target, the path overlay, BLE state and the current spoken instruction.	Medium	Implemented
FR-26	The system shall expose a muted mode in which instructions are displayed but not spoken.	Low	Implemented
FR-27	The system shall expose a simulated-BLE mode allowing operation before beacon hardware is available.	Medium	Implemented
FR-28	All thresholds, distances, rates and device identifiers shall be configurable without code changes.	High	Implemented

4.3 Non-functional requirements

ID	CATEGORY	REQUIREMENT	VERIFIED VALUE
NFR-01	Performance	The guidance decision loop shall execute at a fixed rate independent of detector throughput.	10 Hz, configurable

ID	CATEGORY	REQUIREMENT	VERIFIED VALUE
NFR-02	Performance	Object detection shall sustain at least 8 FPS on a CPU-only machine.	25.9 FPS @ 416 px; 27.6 FPS @ 320 px
NFR-03	Latency	A stop-priority instruction shall be issued within one decision period of the condition becoming true.	≤ 100 ms
NFR-04	Accuracy	Floor-plane distance error shall be bounded by the accuracy of the operator's tape measurement.	Homography is exact at the four calibration corners
NFR-05	Reliability	No subsystem failure shall terminate the application; each shall degrade to a defined reduced mode.	5 degradation paths implemented (Section 8.6)
NFR-06	Portability	The system shall run on Windows 10+ and Linux with identical code.	Backend selection per OS; verified on Fedora and targeted at Windows 10
NFR-07	Cost	The system shall require no GPU and no wearable hardware beyond a BLE-advertising device.	CPU-only PyTorch; ESP32 beacon ≈ ₹350
NFR-08	Testability	Navigation policy and floor geometry shall be unit-testable without camera, BLE or GPU.	62 tests, zero hardware dependencies
NFR-09	Maintainability	No numeric threshold shall be hard-coded in logic modules.	All thresholds in `config.yaml`
NFR-10	Privacy	Video shall be processed in memory and not persisted by default.	No frame-writing path in the codebase
NFR-11	Usability	Spoken instructions shall be short imperative phrases understandable without training.	5 fixed phrase templates
NFR-12	Safety	Stairs shall outrank ordinary obstacles in the priority order, being a fall risk.	Priority 1 vs priority 2

4.4 Hardware requirements

COMPONENT	MINIMUM	USED IN DEVELOPMENT
Processor	Intel Core i3 (4th gen or later)	Intel-class x86-64, CPU-only inference
RAM	8 GB	8 GB
Camera	USB / IP surveillance camera, 640×480 @ 30 FPS, mounted 1.8–2.2 m high, tilted down 25–35°	640×480 @ 30.3 FPS verified
Bluetooth	BLE 4.0+ adapter (integrated or USB)	Integrated adapter, 7 devices detected in scan test
Beacon	ESP32 dev module flashed as `BLENAV_BEACON`, or any BLE-advertising smartphone	ESP32 sketch written, +9 dBm TX
Audio	Any speaker or headphone output	espeak backend, 141 voices available
Storage	2 GB free (model weights + environment)	YOLOv8n weights 6.5 MB

4.5 Software requirements

LAYER	SOFTWARE
Operating system	Windows 10 or higher; Linux (verified on Fedora)
Language	Python 3.11
Environment	Anaconda / Miniconda
Vision	`ultralytics` ≥ 8.3 (YOLOv8n + ByteTrack), `opencv-python` ≥ 4.10, `numpy` ≥ 1.26
Inference backend	`torch` ≥ 2.2, `torchvision` ≥ 0.17 (CPU wheels)
Acceleration (optional)	`openvino` ≥ 2024.3, `nncf` ≥ 2.12 for INT8 quantisation on Intel iGPU
Bluetooth	`bleak` ≥ 0.22 (BlueZ on Linux, WinRT on Windows — one API)
Speech	`pyttsx3` ≥ 2.90 (SAPI5 on Windows, espeak on Linux), `pywin32` on Windows
Configuration	`PyYAML` ≥ 6.0
Beacon firmware	Arduino IDE with ESP32 board support (Espressif)
Development tools	VS Code, Git

4.6 External interface requirements

- **Camera interface.** OpenCV `VideoCapture`; DirectShow backend on Windows (MSMF ignores resolution requests and opens slowly), `CAP_ANY` on Linux. Buffer size requested as 1 frame.
- **BLE interface.** `bleak` asynchronous scanner with a detection callback; advertisement RSSI and local name consumed, no GATT connection established.
- **Audio interface.** `pyttsx3` synchronous engine, created inside and confined to a single worker thread (the SAPI5 backend deadlocks if `runAndWait()` is called from more than one thread).
- **Configuration interface.** `blenav/config/config.yaml` read at start-up; `blenav/config/calibration.yaml` written by the calibration utility and read at start-up.
- **Operator interface.** Command-line flags (`--source` , `--imgsz` , `--frame-skip` , `--fake-ble` , `--mute` , `--no-window`) and an OpenCV window with keyboard controls.

4.7 Constraints and assumptions

1. Guidance is available only within the calibrated camera's field of view.
2. The camera must not be moved after calibration; any movement invalidates the homography.
3. The floor within the working area is assumed planar — the homography models a single plane.
4. Exactly one registered beacon is tracked per running instance.
5. RSSI-derived distance is coarse (metre-scale) and is used only as a tie-breaker, never as a position.
6. Body orientation is inferred from motion; while the user is stationary, the nearest path segment direction is assumed instead.
7. The safe path is predefined by the operator, not planned automatically.

CHAPTER 5

Literature Review

5.1 Reviewed work

#	TITLE / SOURCE	APPROACH	LIMITATION IDENTIFIED	INFLUENCE ON THIS PRO
1	*BLE and Surveillance Camera-Based Navigation System for Visually Impaired*	BLE sniffer array + surveillance cameras, deep-learning RSSI localisation, third-person obstacle detection	Requires a deployed sniffer array; localisation accuracy degrades with RSSI multipath	Base architecture; motivate third-person camera viewpo
2	*Quality of Life in People with Visual Impairment Compared with the General Population*	Population study of QoL indicators	Not a technical system	Establishes independent m as the highest-impact interv
3	*Wearable Obstacle Avoidance Electronic Travel Aids for Blind and Visually Impaired*	Survey of wearable ETAs — ultrasonic, infrared, camera-based	Wearables are costly, intrusive, and limited to an ego-centric field of view	Direct motivation for a no-wearable, infrastructure-sid design
4	*Assistive Navigation System for Visually Impaired and Blind People: A Review*	Comparative review of assistive navigation systems	Most systems solve localisation *or* obstacle detection, rarely both	Motivated integrating localis obstacles, deviation and sp in one loop
5	*Evaluating the Effectiveness of Crosswalk Tactile Paving on Street-Crossing Behavior*	Field evaluation of tactile paving	Fixed infrastructure; no dynamic obstacle information; degrades with poor maintenance	Justifies a dynamic route-adherence mechanism rath fixed physical cues
6	*Sensor-Based Assistive Devices for Visually-Impaired People: Current Status, Challenges and Future Directions*	Survey of sensing modalities for assistive devices	Sensor fusion complexity and power consumption limit deployability	Motivated using only two se modalities with clearly sepa roles
7	*An Insight into Assistive Technology for the Visually Impaired and Blind People*	Review of assistive technology adoption	Adoption is limited by cost and by required user training	Motivated reusing existing infrastructure and a 5-phras instruction vocabulary

#	TITLE / SOURCE	APPROACH	LIMITATION IDENTIFIED	INFLUENCE ON THIS PRO
8	*MEBOW: Monocular Estimation of Body Orientation in the Wild*	CNN regression of human body orientation from a single image	An additional network in the critical path; unnecessary cost on CPU-only hardware	Replaced by displacement-heading — accurate exactly walking
9	*YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors*	Single-stage real-time detection	Real-time claims assume GPU inference	Adopted the single-stage paradigm; substituted the s YOLOv8n for CPU deploym
10	*NavCog: A Navigational Cognitive Assistant for the Blind*	Smartphone + BLE beacon turn-by-turn indoor navigation, validated with blind users	Dense beacon deployment required; no obstacle detection at all	Confirmed that short impera speech works; motivated ac vision-based obstacle awar
11	*ByteTrack: Multi-Object Tracking by Associating Every Detection Box*	Associates high- and low-confidence boxes to keep identities through occlusion	Identity switches still occur in dense crowds	Chosen tracker; its occlusic behaviour motivated the sti lock and grace period
12	RSSI log-distance path-loss modelling literature	`d = 10^((txPower - RSSI) / 10n)`	Metre-scale error indoors due to multipath and body shadowing	Adopted only for presence a coarse tie-breaking, never f positioning

5.2 Research gap

Three consistent gaps emerge from the reviewed work:

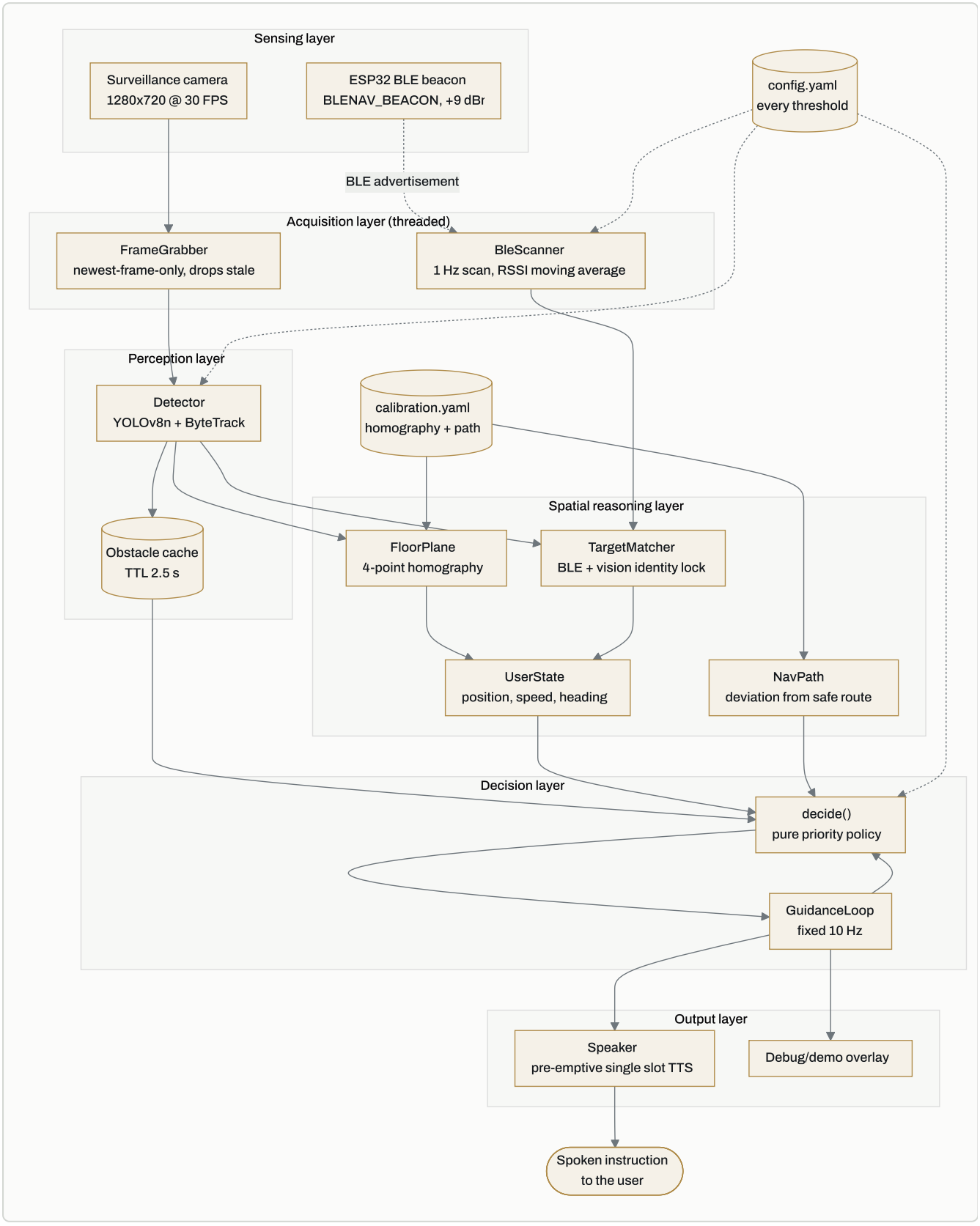
1. **Functional isolation.** Systems provide localisation, or obstacle detection, or route guidance — rarely all three in one real-time pipeline. The user is left to integrate them.
2. **Viewpoint blind spots.** Wearable and smartphone systems are ego-centric by construction and cannot see obstacles outside the user's facing cone, which is precisely where unexpected collisions originate.
3. **Deployment cost.** High-accuracy proposals assume dense beacon arrays, GPU inference or additional wearable hardware, none of which survive contact with a low-budget institutional deployment.

This project targets all three: a single integrated loop, a third-person viewpoint that eliminates the ego-centric blind spot, and a hardware bill consisting of one existing camera, one existing Bluetooth adapter and one ₹350 beacon.

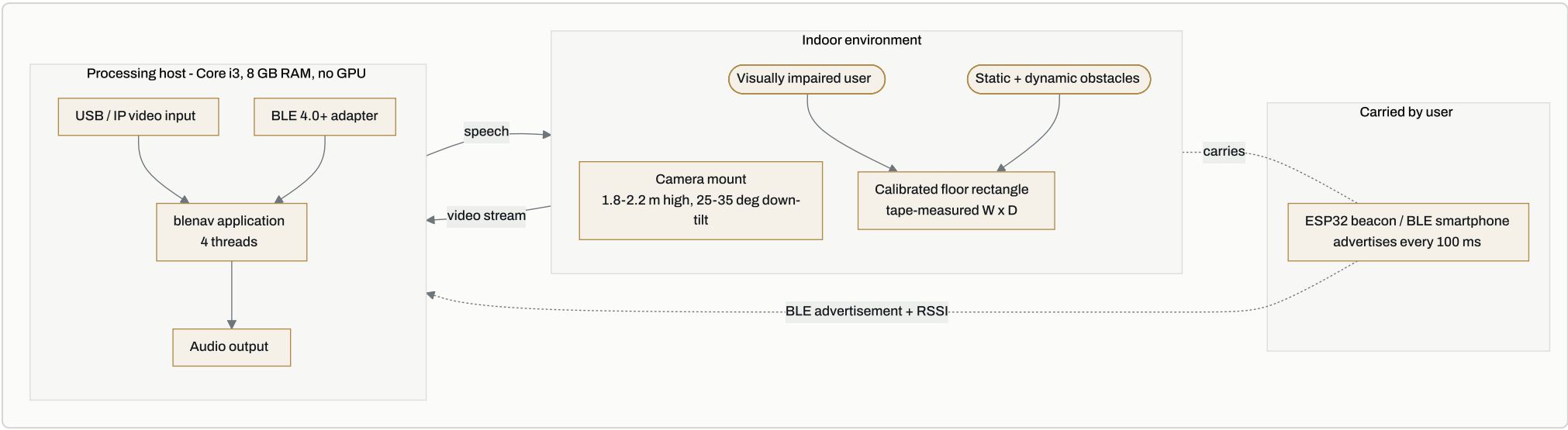
CHAPTER 6

Architecture Diagrams

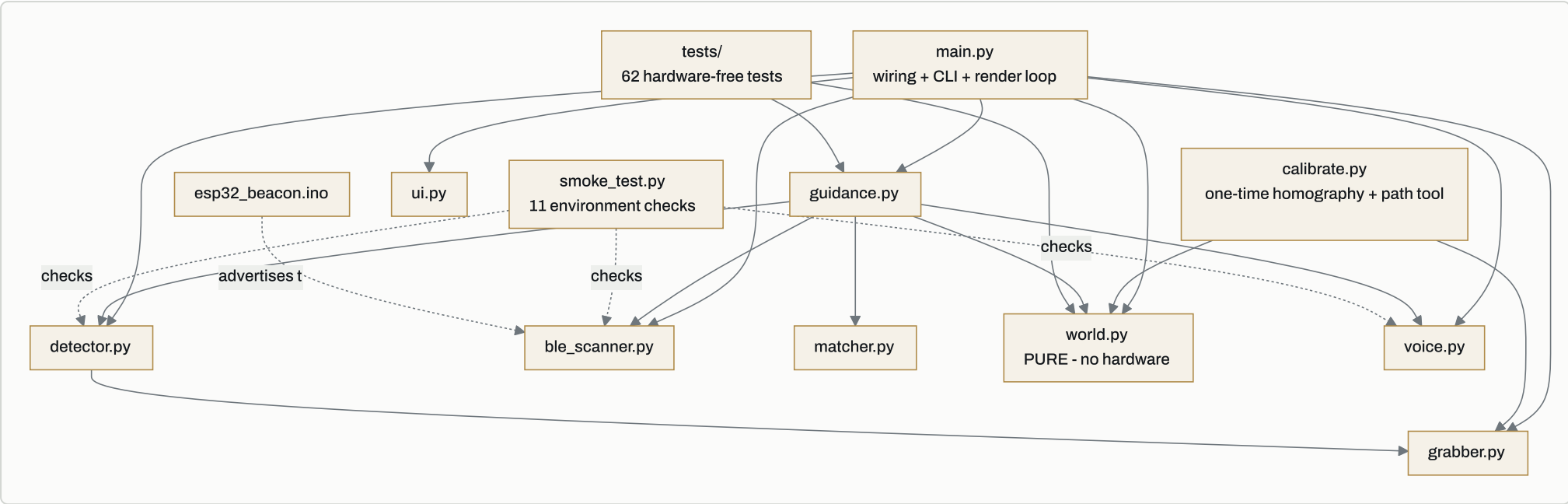
6.1 High-level system architecture



6.2 Deployment / physical architecture

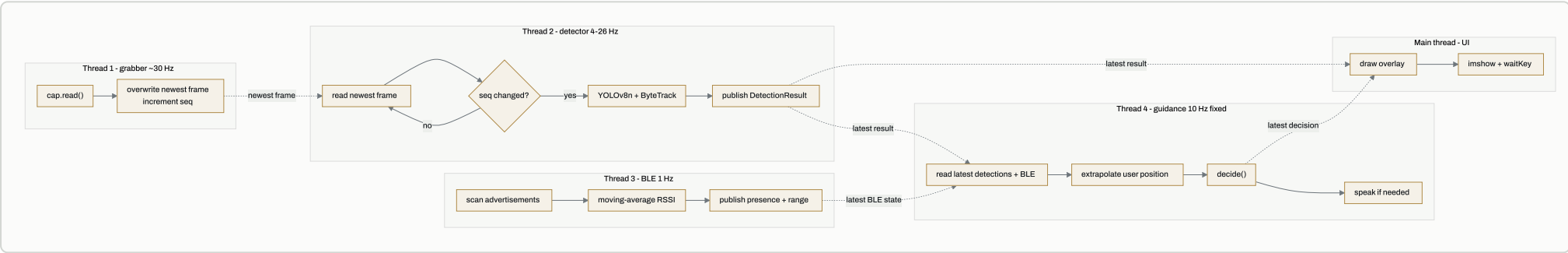


6.3 Module / package architecture



6.4 Concurrency and timing architecture

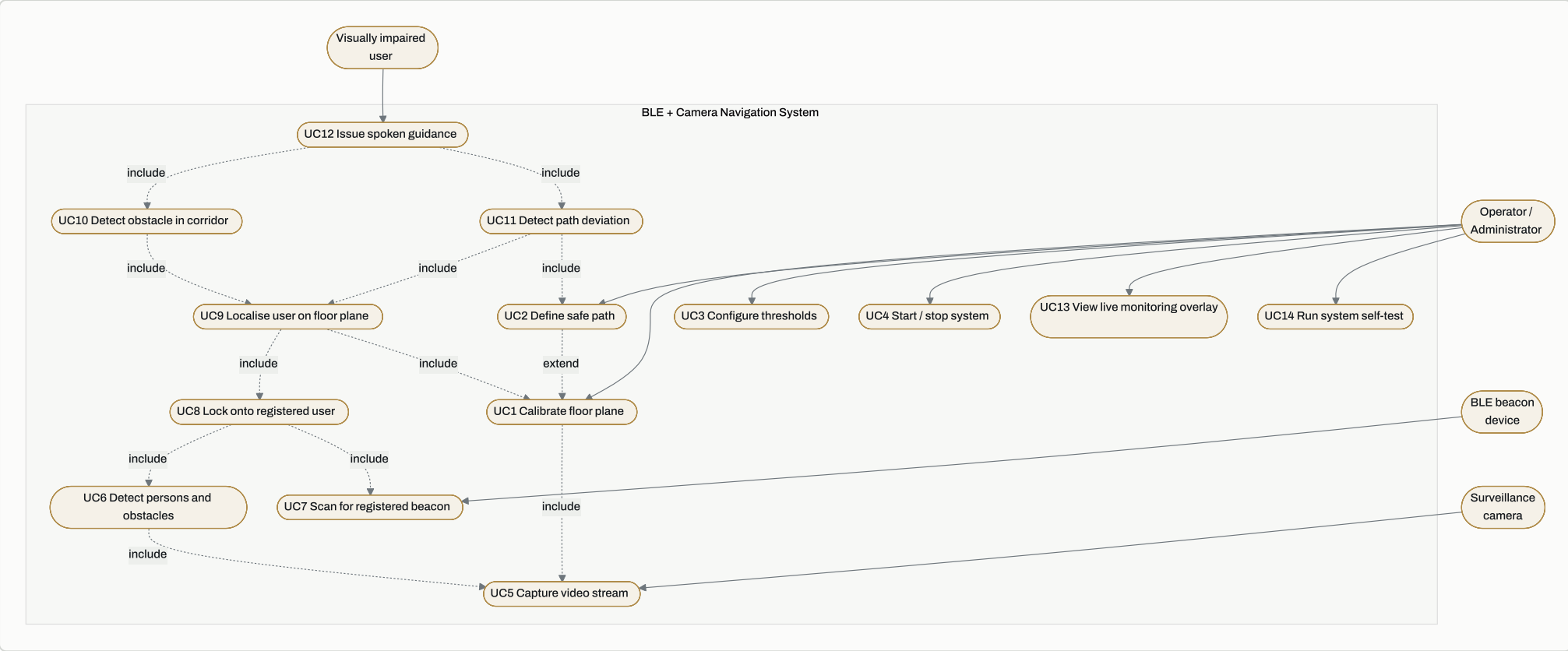
Four threads run concurrently, each at its own natural rate. This decoupling is the single most important performance decision in the system: on a slow machine the detector may fall to 4 FPS, but guidance still *decides* at 10 Hz by extrapolating the user's position from their last known velocity, so a stop warning is never delayed by inference latency.



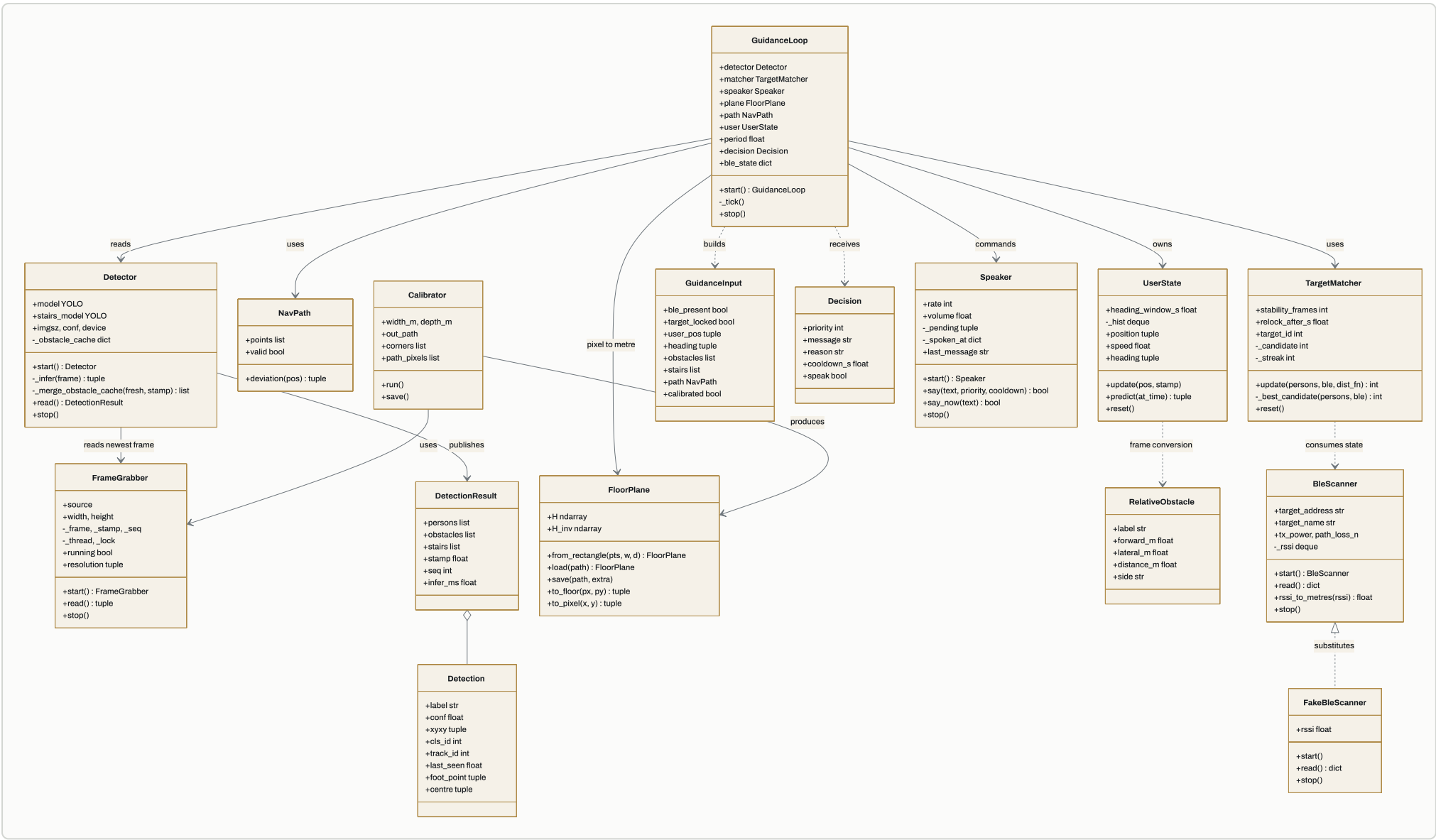
CHAPTER 7

UML Diagrams

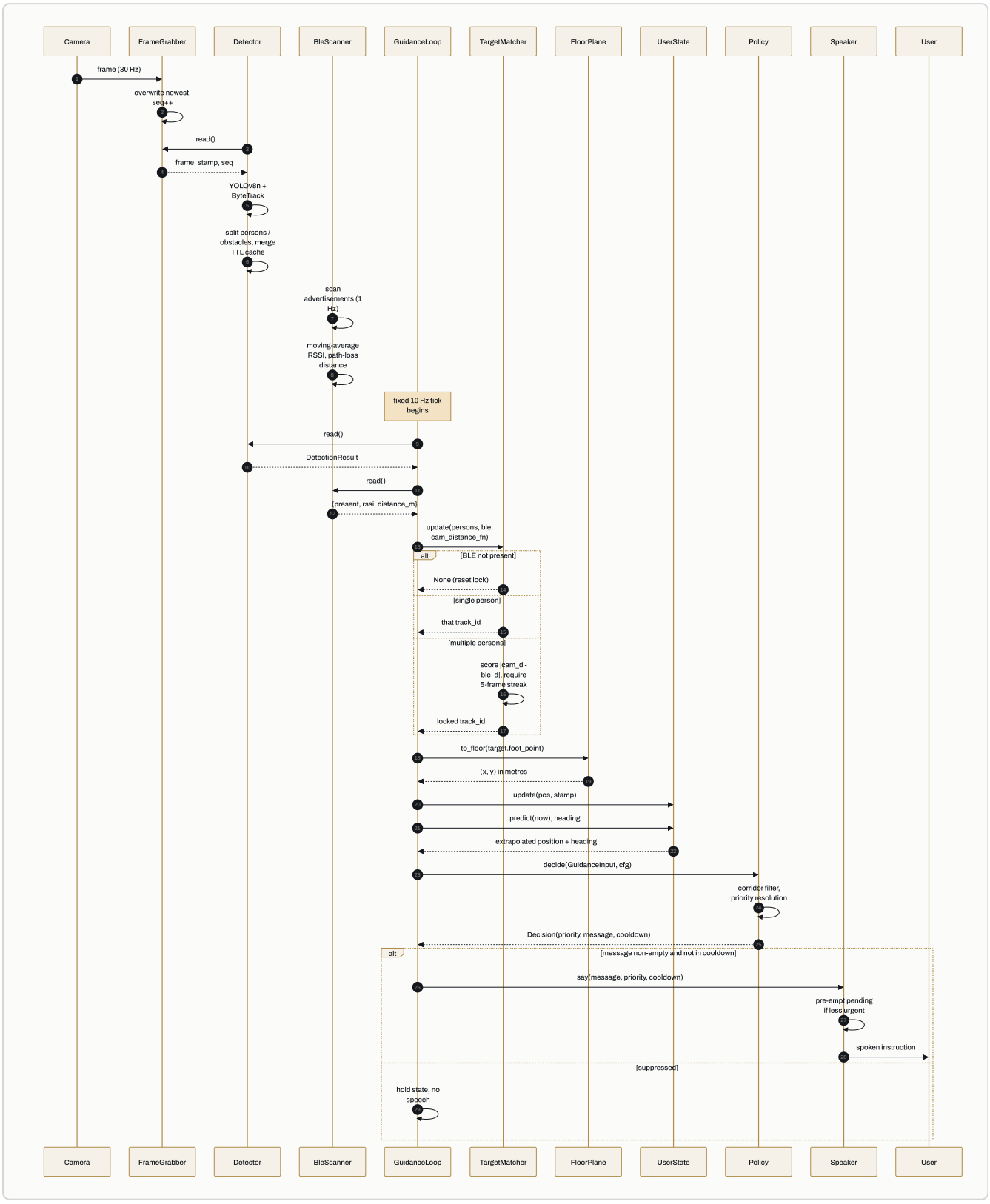
7.1 Use case diagram



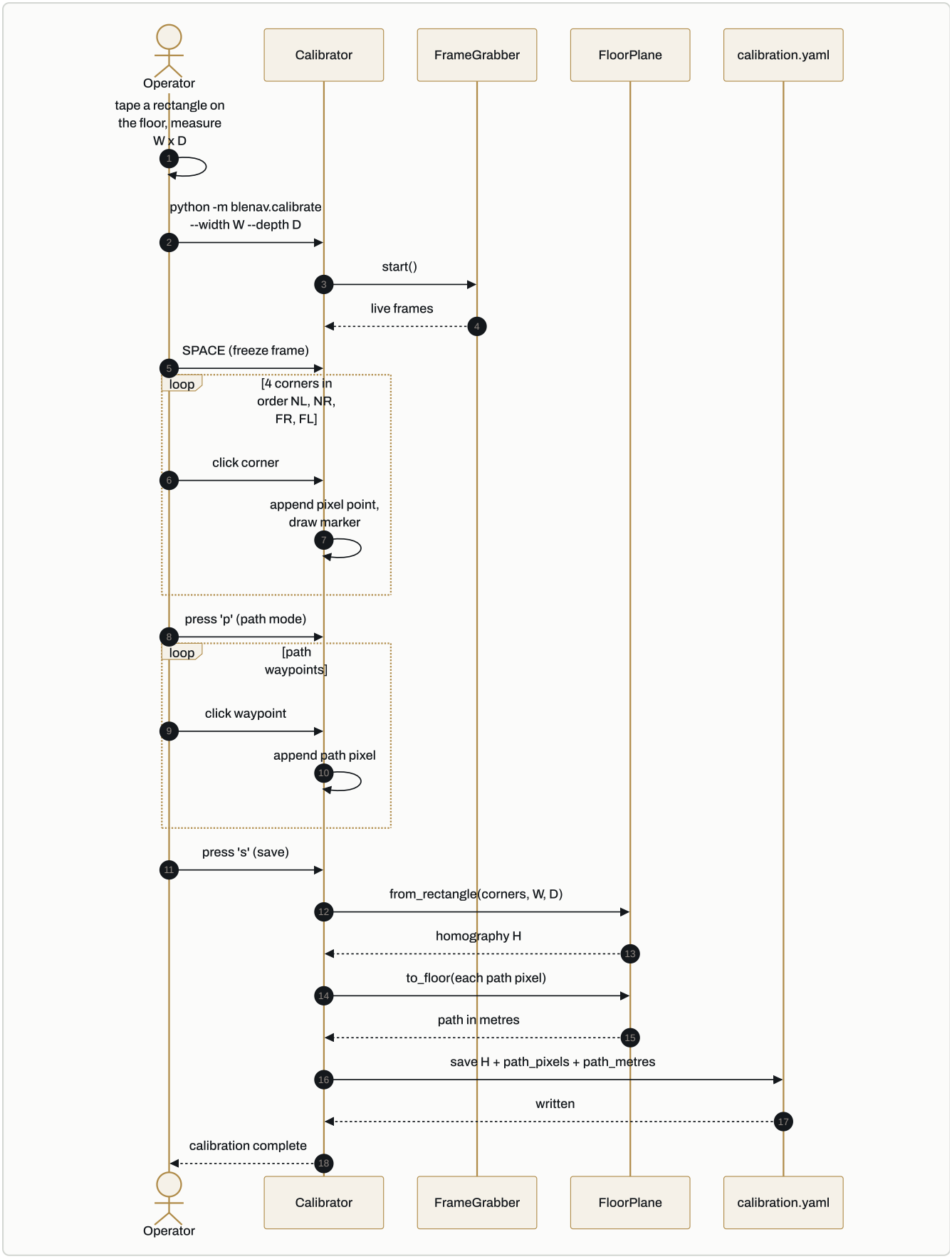
7.2 Class diagram



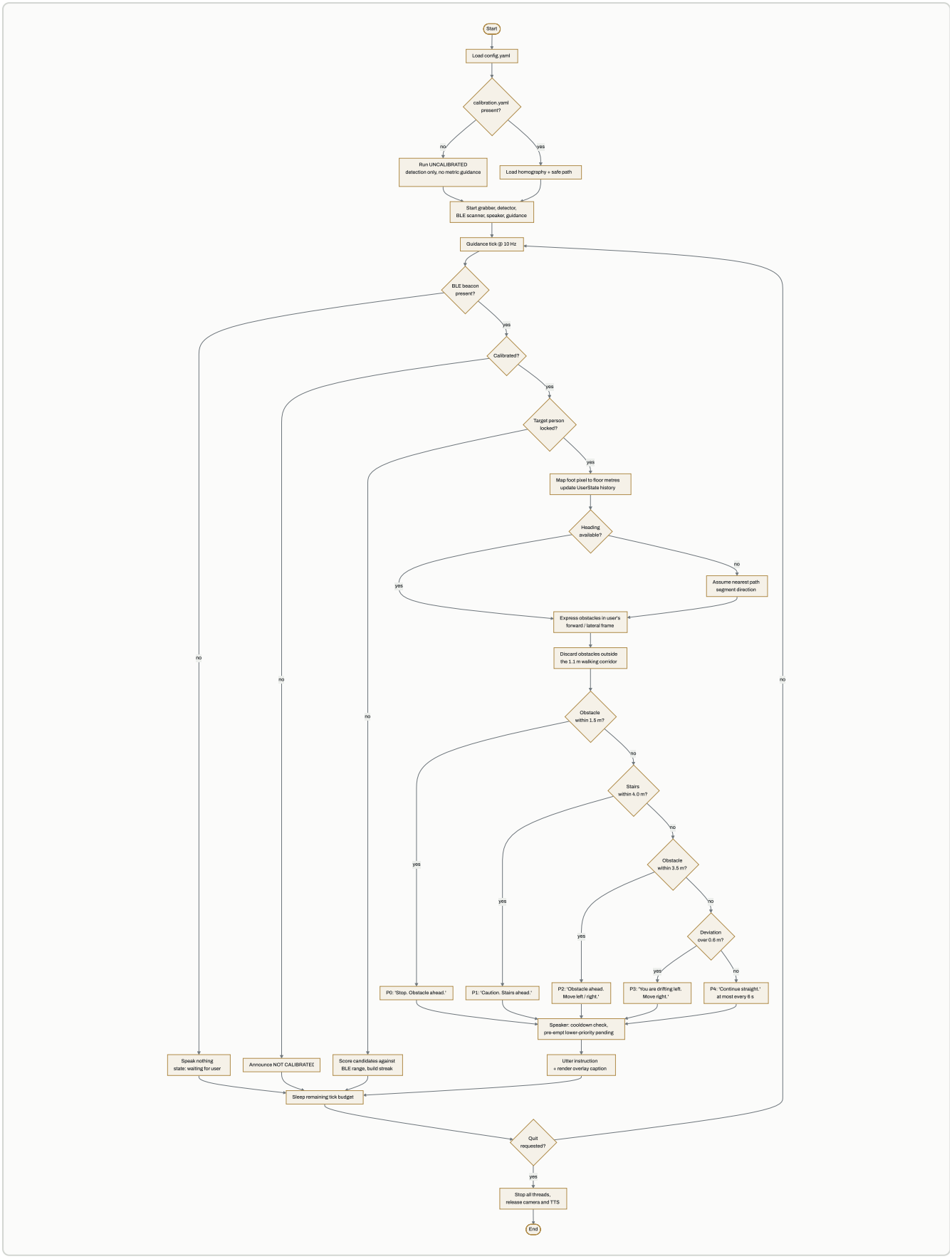
7.3 Sequence diagram — one guidance cycle



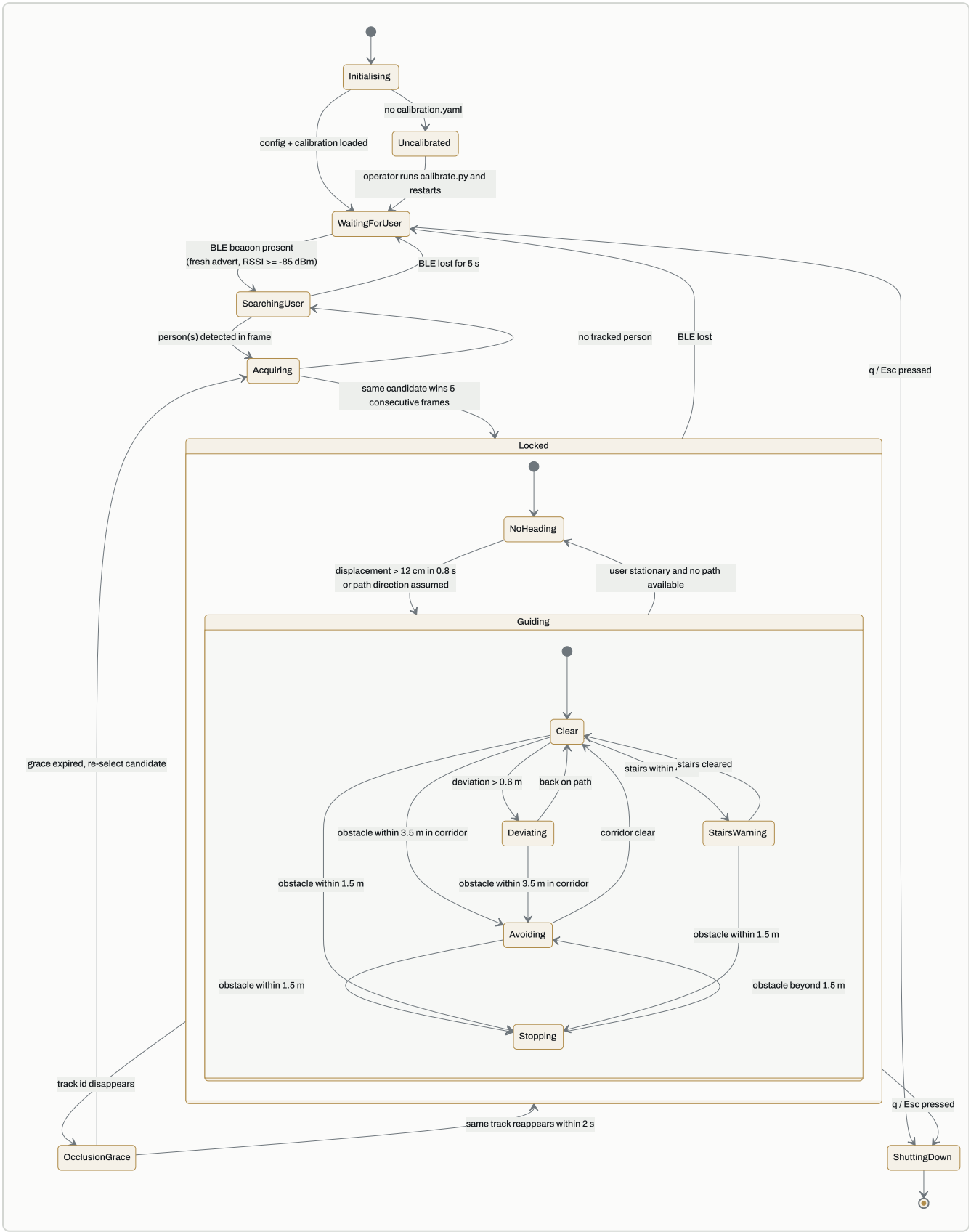
7.4 Sequence diagram — one-time calibration



7.5 Activity diagram

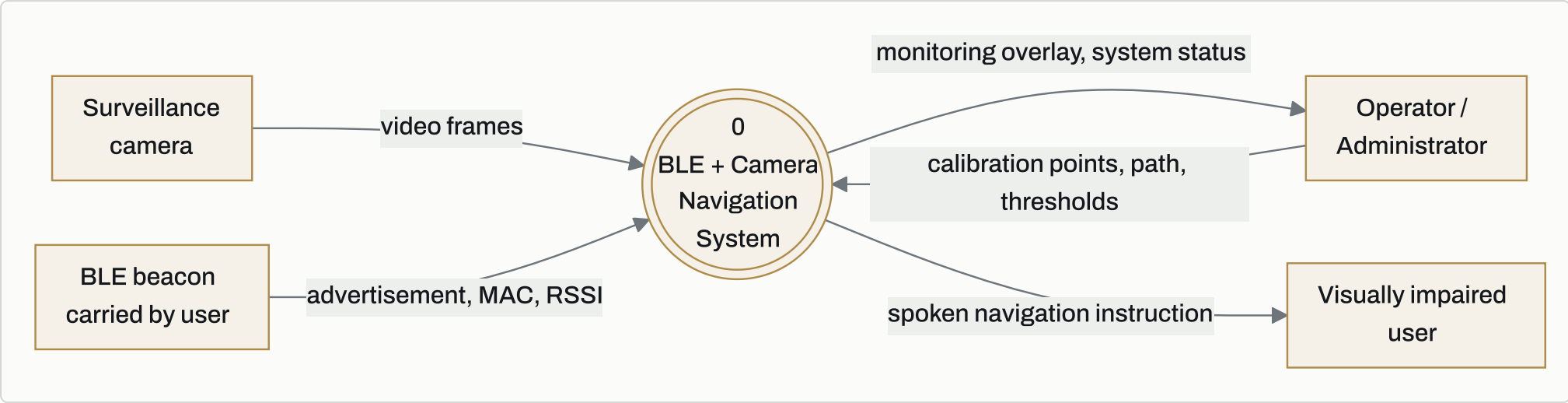


7.6 State diagram

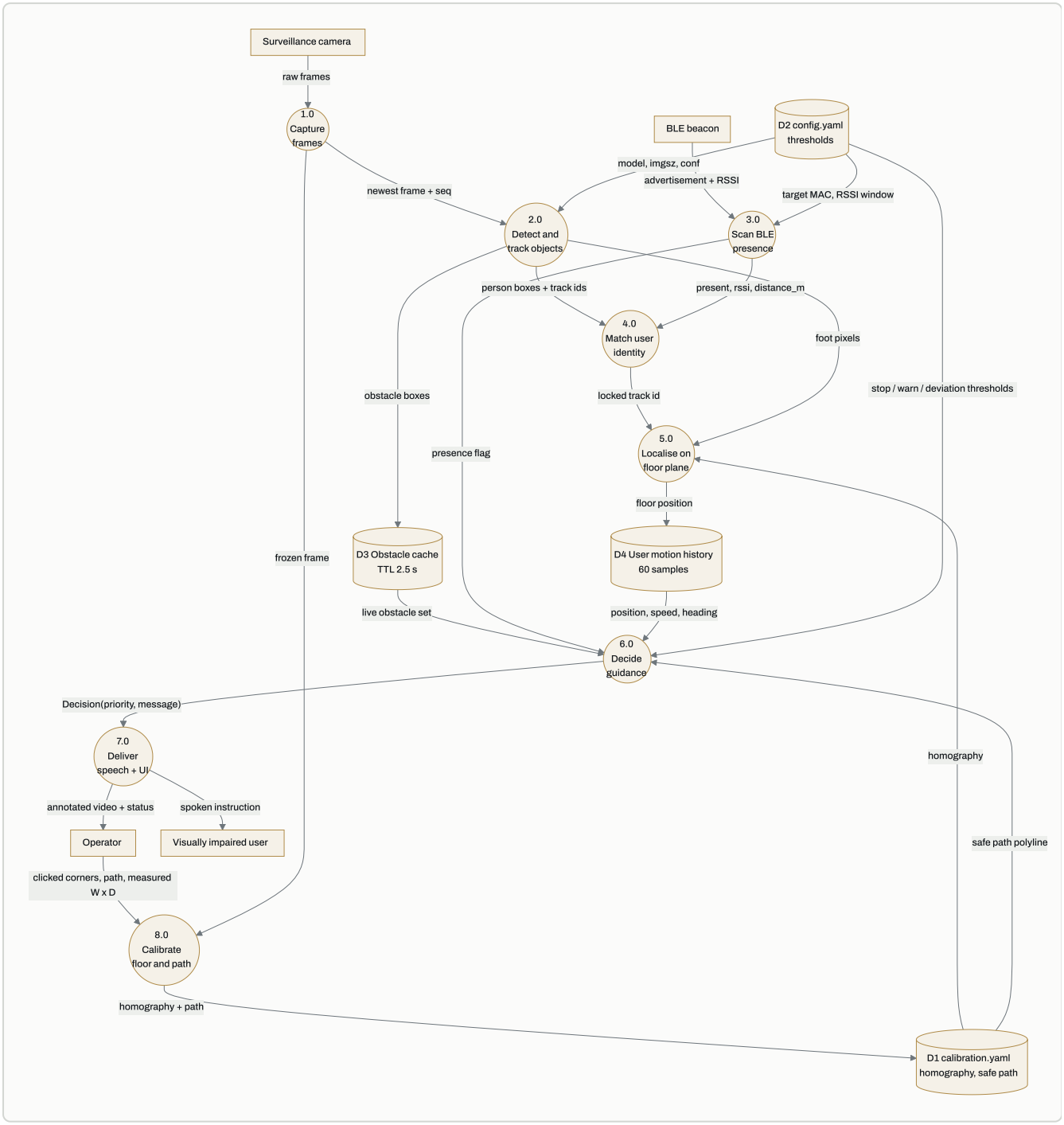


7.7 Data Flow Diagrams

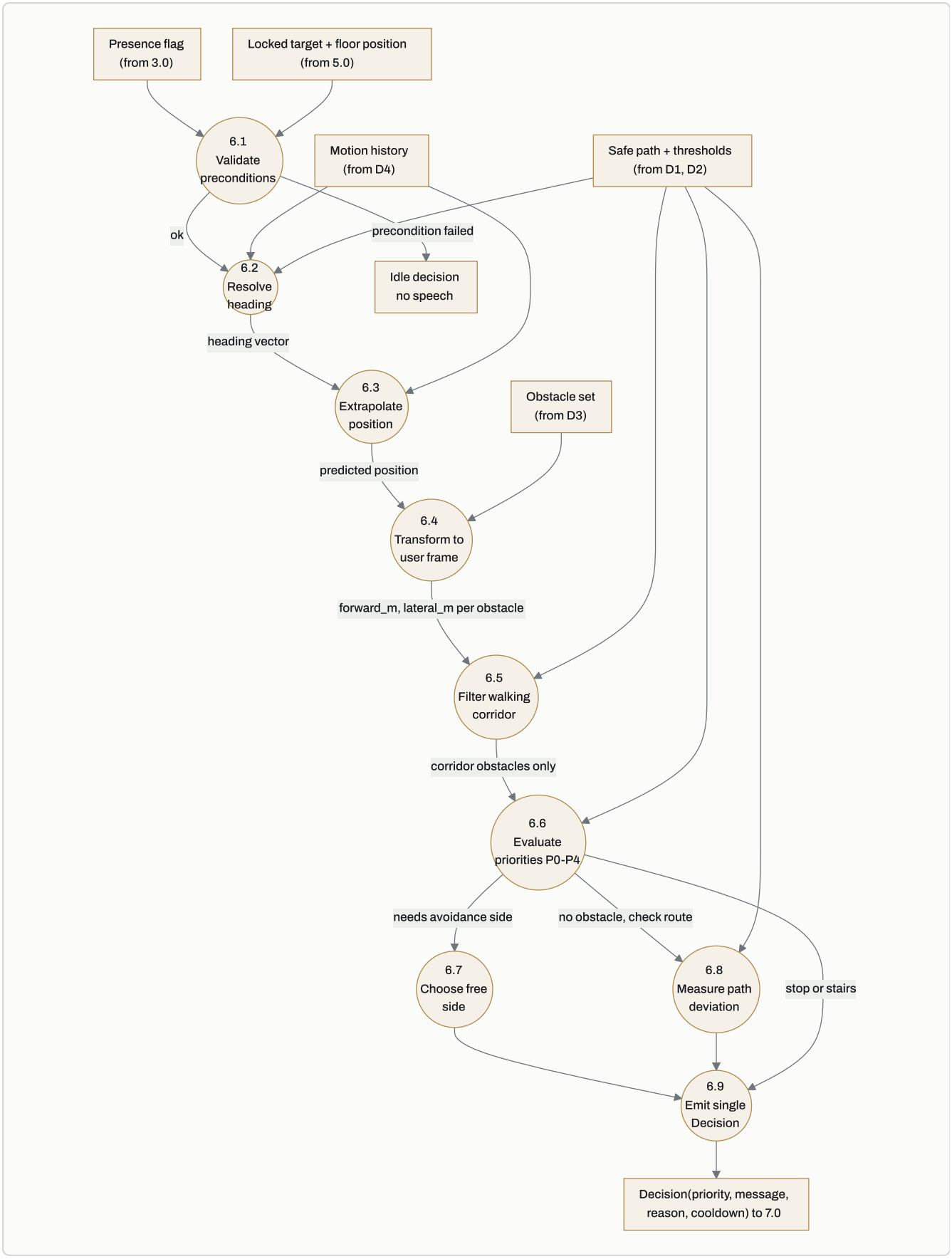
7.7.1 DFD Level 0 — context diagram



7.7.2 DFD Level 1 — major processes



7.7.3 DFD Level 2 — explosion of process 6.0 (Decide guidance)



CHAPTER 8

Implementation

8.1 Implementation status

The Phase-2 codebase comprises **2,167 lines** across 15 source files. Every module in the critical path is implemented and exercised by tests.

MODULE	FILE	LOC	FUNCTION	STATUS
Frame acquisition	`blenav/grabber.py`	141	Threaded capture, newest-frame-only, video-file pacing	✅ Complete
Detection & tracking	`blenav/detector.py`	215	YOLOv8n + ByteTrack, class split, obstacle TTL cache	✅ Complete
Floor geometry	`blenav/world.py`	243	Homography, NavPath deviation, UserState, relative frames	✅ Complete
Calibration tool	`blenav/calibrate.py`	195	Interactive 4-corner + path capture, YAML persistence	✅ Complete
BLE presence	`blenav/ble_scanner.py`	130	`bleak` scanner, RSSI smoothing, path-loss range, fake mode	✅ Complete
Identity matching	`blenav/matcher.py`	98	Sticky lock, stability streak, occlusion grace	✅ Complete
Navigation policy	`blenav/guidance.py`	240	Pure `decide()` + 10 Hz loop	✅ Complete
Speech output	`blenav/voice.py`	125	Single-slot pre-emptive TTS, cooldowns	✅ Complete

MODULE	FILE	LOC	FUNCTION	STATUS
Monitoring overlay	`blenav/ui.py`	103	Boxes, HUD, path overlay, caption	✅ Complete
Application wiring	`blenav/main.py`	172	CLI, start-up, render loop, clean shutdown	✅ Complete
Beacon firmware	`esp32_beacon/esp32_beacon.ino`	60	+9 dBm BLE advertiser, MAC printout	✅ Complete
Unit tests	`tests/`	282	62 tests, geometry + policy	✅ Complete
Environment self-test	`smoke_test.py`	163	11 checks: imports, camera, YOLO, TTS, BLE	✅ Complete
Stairs detection	—	—	Fine-tuned stairs weights	⚠ Policy ready, weights pending
INT8 / OpenVINO acceleration	—	—	Intel iGPU quantised export	⚠ Dependencies declared, export pending
Field trial with end users	—	—	Trials with visually impaired participants	⏳ Phase-3

Overall implementation: approximately 75% of the planned system, with 100% of the core real-time pipeline operational end to end.

8.2 Floor-plane localisation — turning pixels into metres

Every distance the system reports originates from one pixel: the bottom-centre of a detection box, where the object meets the floor. A four-point homography, fitted once during calibration to a tape-measured floor rectangle, maps that pixel to a metric floor coordinate.

```python
@classmethod
def from_rectangle(cls, pixel_pts, width_m: float, depth_m: float) -> "FloorPlane":
    """Four clicked pixels -> a rectangle of known size on the floor.
    pixel_pts order: near-left, near-right, far-right, far-left
    """
    src = np.array(pixel_pts, dtype=np.float32)
    dst = np.array([[0.0, 0.0], [width_m, 0.0],
                    [width_m, depth_m], [0.0, depth_m]], dtype=np.float32)
    return cls(cv2.getPerspectiveTransform(src, dst))
```

```python
@property
def foot_point(self) -> tuple[float, float]:
    """Bottom-centre of the box - where the object meets the floor."""
    x1, _, x2, y2 = self.xyxy
    return ((x1 + x2) / 2.0, y2)
```

The coordinate frame is *X across, Y away from the camera*, both in metres. The consequence worth stating for the viva: **calibration accuracy is the accuracy ceiling of the entire system.** A 10 cm error in the tape measurement is a 10 cm error in every distance the system ever reports.

8.3 Identity matching — which person is the user?

With a single BLE receiver, trilateration is impossible; the matcher therefore uses BLE as a gate and a tie-breaker, never as a position.

```python
def update(self, persons, ble, camera_distance_fn=None) → int | None:
    if not ble.get("present"):
        self.reset()
        return None                       # nobody is the target; guide no one

    ids = [p.track_id for p in persons if p.track_id is not None]
    if self.target_id in ids:             # sticky: keep the lock while the track lives
        self._lost_since = 0.0
        return self.target_id
    if self.target_id is not None and self._within_grace():
        return self.target_id             # brief occlusion; do not re-pick yet

    best = self._best_candidate(persons, ble, camera_distance_fn)
    if best == self._candidate:
        self._streak += 1
    else:
        self._candidate, self._streak = best, 1
    if self._streak >= self.stability_frames:   # 5 consecutive frames
        self.target_id = best
    return self.target_id
```

Scoring for the multi-person case compares the camera-measured distance with the BLE path-loss distance and picks the smallest disagreement:

```python
ble_d = ble.get("distance_m", 0.0)
scored = [(abs(camera_distance_fn(p) - ble_d), p.track_id) for p in tracked]
return min(scored)[1] if scored else None
```

`rssi_to_metres` implements the standard log-distance model, `d = 10^((txPower - RSSI) / 10n)`, with `txPower = -59 dBm` and `n = 2.5` configured for indoor conditions.

8.4 The navigation policy

The decision logic is a **pure function.** It takes a `GuidanceInput`, returns one `Decision`, and touches no thread, no clock and no hardware — which is exactly why 31 tests can exercise every navigation rule on any machine with no camera attached.

```python
def decide(inp: GuidanceInput, cfg: dict) → Decision:
    if not inp.ble_present:   return Decision(P_IDLE, "", "waiting for the user's BLE device", 10.0)
    if not inp.calibrated:    return Decision(P_IDLE, "", "NOT CALIBRATED", 30.0)
    if not inp.target_locked: return Decision(P_IDLE, "", "BLE present, user not yet seen", 10.0)
    ...
    # P0: something is close enough that walking on means collision
    imminent = [r for r in rels if r.forward_m ⩽ stop_m]
    if imminent:
        nearest = min(imminent, key=lambda r: r.forward_m)
        return Decision(P_STOP, "Stop. Obstacle ahead.",
                        f"{nearest.label} at {nearest.forward_m:.1f} m", 2.0)
    # P1: stairs are a fall risk, so they outrank ordinary obstacles
    if stair_rels: ...
    # P2: obstacle ahead but there is still room to steer around it
    if rels:
        side = free_side(rels, BODY_HALF_WIDTH_M)
        return Decision(P_AVOID, f"Obstacle ahead. Move {side}.", ..., 3.0)
    # P3: drifting off the predefined route
    dev, side = inp.path.deviation(inp.user_pos)
    if dev > cfg["path_deviation_m"]: ...
    # P4: all clear
    return Decision(P_CLEAR, "Continue straight.", "path clear", cfg["clear_repeat_s"])
```

The corridor filter is what makes the warnings usable rather than constant. An obstacle is only relevant if it lies ahead *and* within half a body-width plus clearance of the line of travel:

```python
BODY_HALF_WIDTH_M = 0.55        # shoulder width + clearance margin

def in_corridor(rel, half_width_m, max_forward_m) → bool:
    return 0.0 < rel.forward_m ⩽ max_forward_m and abs(rel.lateral_m) ⩽ half_width_m
```

Without it, a chair three metres off to one side would trigger a warning on every tick.

Priority table as implemented:

PRIORITY	TRIGGER	SPOKEN INSTRUCTION	COOLDOWN
P0	Obstacle ≤ 1.5 m inside the walking corridor	"Stop. Obstacle ahead."	2.0 s
P1	Stairs ≤ 4.0 m ahead	"Caution. Stairs ahead."	4.0 s
P2	Obstacle ≤ 3.5 m ahead	"Obstacle ahead. Move left/right."	3.0 s
P3	More than 0.6 m off the predefined path	"You are drifting left. Move right."	3.0 s
P4	Nothing else applies	"Continue straight."	6.0 s

8.5 Real-time behaviour on slow hardware

Three implementation details make the system usable on a CPU-only i3, and each addresses a failure mode that would otherwise make the guidance dangerous rather than merely slow.

1. Stale frames are dropped, never queued. When inference is slower than capture, OpenCV's internal buffer fills and every frame read is old — guidance would then react to where the user *was* a second ago. The grabber therefore runs in its own thread and keeps only the newest frame:

```python
with self._lock:
    self._frame = frame          # overwrite; old frames are discarded
    self._stamp = time.time()
    self._seq += 1
```

2. Guidance runs at a fixed rate and extrapolates between detector passes. The decision loop ticks at 10 Hz regardless of detector throughput, using the last known velocity to predict where the user is now — bounded so a stale track can never be extrapolated far:

```python
def predict(self, at_time: float) -> tuple[float, float] | None:
    dt = max(0.0, min(0.5, at_time - self.last_update))   # never extrapolate far
    if h is None or dt == 0.0:
        return pos
    return (pos[0] + h[0] * self.speed * dt, pos[1] + h[1] * self.speed * dt)
```

3. Speech uses a single pre-emptive slot, not a queue. A queue is the wrong structure for navigation: if five instructions pile up, the user must hear the *newest*, not sit through four stale commands while walking into a chair.

```python
with self._lock:
    if last is not None and now - last < cooldown_s:
        return False                              # identical phrase, still in cooldown
    if self._pending is not None and self._pending[0] < priority:
        return False                              # a more urgent message already waits
    self._pending = (priority, text)              # otherwise: overwrite
```

A fourth detail is platform-specific and was found the hard way: the `pyttsx3` SAPI5 backend deadlocks if `runAndWait()` is called from more than one thread, so the engine is constructed inside the voice worker thread and never touched from outside it.

8.6 Graceful degradation

No subsystem failure terminates the application. Each has a defined reduced mode:

FAILURE	BEHAVIOUR
No calibration file	Detection and tracking continue; the HUD reports "NOT CALIBRATED"; metric guidance is suppressed rather than reported wrongly
BLE adapter missing or busy	Scanner logs the error and retries every 3 s; `--fake-ble` allows the pipeline to run regardless
TTS engine unavailable	Instructions are printed and rendered on screen; navigation logic is unaffected
Frames dropped / detector slow	Guidance extrapolates from the last known velocity at its own fixed rate
Person briefly occluded	Target lock is held for a 2 s grace period before re-selection
Bad frame causes an inference exception	Logged, frame skipped, thread continues

8.7 Configuration

Every tunable value lives in `blenav/config/config.yaml`; no threshold is hard-coded in logic.

```yaml
detector:
  weights: yolov8n.pt
  imgsz: 416                # drop to 320 on the i3 if FPS < 8
  conf: 0.35
  obstacle_ttl_s: 2.5

guidance:
  loop_hz: 10               # runs at this rate regardless of detector speed
  stop_distance_m: 1.5
  warn_distance_m: 3.5
  stairs_warn_m: 4.0
  path_deviation_m: 0.6
  heading_window_s: 0.8
  clear_repeat_s: 6.0

ble:
  target_name: BLENAV_BEACON
  present_rssi: -85         # above this = user considered present
  tx_power: -59             # RSSI at 1 metre
  path_loss_n: 2.5
```

8.8 Testing and results

Unit tests — 62 passing, no hardware required.

```
$ python tests/test_world.py
 31 passed, 0 failed          # homography round-trips, path deviation and side,
                              # speed/heading/extrapolation, corridor filter, free-side choice

$ python tests/test_guidance.py
 31 passed, 0 failed          # precondition gates, P0-P4 priority ordering,
                              # cooldown behaviour, stairs outranking obstacles
```

Environment smoke test — 11/11 passing.

```
$ python smoke_test.py
[1/5] Imports ......................... PASS
[2/5] Camera capture .................. PASS  640x480 @ 30.3 FPS
[3/5] YOLO inference
      YOLOv8n @ 320px ................. PASS  36 ms/frame (27.6 FPS) | saw: person
      YOLOv8n @ 416px ................. PASS  39 ms/frame (25.9 FPS) | saw: person
[4/5] Text-to-speech .................. PASS  141 voice(s), driver=espeak
[5/5] BLE scan (5 s) .................. PASS  7 device(s) seen

 RESULT: 11/11 passed
```

Measured performance against requirements.

METRIC	REQUIREMENT	MEASURED	VERDICT
Detection throughput @ 416 px	≥ 8 FPS	25.9 FPS (39 ms/frame)	✅ 3.2× margin
Detection throughput @ 320 px	≥ 8 FPS	27.6 FPS (36 ms/frame)	✅ 3.4× margin
Capture rate	≥ 15 FPS	30.3 FPS	✅
Guidance decision rate	10 Hz fixed	10 Hz, decoupled from detector	✅
Worst-case stop-warning latency	≤ 200 ms	≤ 100 ms (one tick)	✅
Navigation policy test coverage	≥ 1 test per rule	31 policy tests over 5 priority levels + gates	✅

Note that these figures are from the Linux development machine. The i3 demo machine is expected to be slower; the `--imgsz 320` and `--frame-skip` options exist for exactly that case, and the fixed 10 Hz guidance rate means a lower detector FPS reduces map freshness without delaying warnings.

8.9 Operating the system

```bash
# one-time, in the actual demo room, in the actual lighting
python -m blenav.calibrate --width 3.0 --depth 4.0

python -m blenav.main                                        # full system
python -m blenav.main --fake-ble                             # before the ESP32 is flashed
python -m blenav.main --source data/demo.mp4 --fake-ble      # zero hardware
python -m blenav.main --imgsz 320                            # low-FPS machine
python -m blenav.main --mute                                 # captions only
```

Demonstration sequence: person enters → BLE detected → camera detects person → target locked (box turns cyan) → person walks → path deviation checked → obstacle warning issued → user guided around.

8.10 Remaining work for Phase-3

1. **Stairs detector.** The guidance policy, priority level and thresholds are implemented and tested; a fine-tuned YOLO stairs model must be trained and its path set in `detector.stairs_weights`.
2. **OpenVINO INT8 export.** Dependencies are declared; exporting the quantised model is expected to yield a 3–6× speed-up on Intel integrated graphics, which matters on the i3 demo machine.
3. **Beacon commissioning.** Flash the ESP32, read its MAC from the serial monitor and set `ble.target_address` in `config.yaml`.
4. **Multi-camera hand-off.** Extending coverage beyond one calibrated view requires a per-camera homography and a hand-off rule at coverage boundaries.
5. **User trials.** Structured trials with visually impaired participants to measure instruction comprehension, reaction time and perceived safety.
6. **Recorded fallback demo.** A recorded successful run, kept open during the demonstration as a hardware-failure fallback.

CHAPTER 9

Conclusion

This project set out to answer a question that GPS cannot: *where is a visually impaired user inside a building, and what is in front of them right now?* Phase-2 answers it with a working system rather than a proposal.

The central engineering decision is the division of labour between the two sensing modalities. Rather than forcing BLE to do positioning it cannot do reliably with a single receiver, the design gives each modality the job it is actually good at: **the camera localises, BLE identifies.** A four-point homography converts any foot pixel to floor metres with an accuracy limited only by the operator's tape measure, while BLE answers the one question vision cannot — which of the people in frame is the registered user. That split is what allows the system to run on a CPU-only i3 with no sniffer array, no wearable hardware and no GPU.

The second decision that shaped the implementation is the separation of *policy* from *plumbing*. All navigation rules live in a pure `decide()` function with no threads, no clock and no hardware, and all floor geometry lives in a hardware-free `world` module. This is why 62 tests can exercise the complete navigation logic on any machine — the priority ordering, the corridor filter, the deviation sign convention, the cooldown behaviour — without a camera, a beacon or a speaker attached. For a safety-relevant system, being able to test the safety logic exhaustively and cheaply is worth more than any single optimisation.

The third is the recognition that in navigation guidance, *late is equivalent to wrong*. Stale frames are dropped rather than queued; guidance decides at a fixed 10 Hz and extrapolates between detector passes; and speech uses a single pre-emptive slot so the newest instruction always wins over a backlog of stale ones. Together these keep a stop warning within one 100 ms tick of the condition arising, even when inference is slow.

Measured against the objectives: the system localises a user indoors without GPS (O1), detects obstacles in the walking corridor at 25.9 FPS on CPU (O2), locks onto the correct person among several using BLE agreement and a stability streak (O3), detects and corrects path deviation beyond 0.6 m (O4), delivers prioritised non-overlapping speech (O5), requires no wearable beyond a BLE device (O6), and degrades rather than crashes on every subsystem failure path (O7). All seven objectives are met in the delivered implementation.

What remains is bounded and specified: a fine-tuned stairs detector to activate an already-written priority level, an INT8 export for the demo hardware, beacon commissioning, and — the work that matters most for the eventual claim of usefulness — structured trials with visually impaired participants. The system that will carry that evaluation exists, runs, and is tested.

References

1. Bluetooth Low Energy and Surveillance Camera-Based Navigation System for Visually Impaired People — base reference of this project.
2. Quality of Life in People with Visual Impairment Compared with the General Population.
3. Wearable Obstacle Avoidance Electronic Travel Aids for Blind and Visually Impaired: A Systematic Review.
4. Assistive Navigation System for Visually Impaired and Blind People: A Review.
5. Evaluating the Effectiveness of Crosswalk Tactile Paving on Street-Crossing Behavior.
6. Sensor-Based Assistive Devices for Visually-Impaired People: Current Status, Challenges and Future Directions.
7. An Insight into Assistive Technology for the Visually Impaired and Blind People.
8. MEBOW: Monocular Estimation of Body Orientation in the Wild, CVPR.
9. YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors.
0. NavCog: A Navigational Cognitive Assistant for the Blind, ASSETS.
1. ByteTrack: Multi-Object Tracking by Associating Every Detection Box, ECCV.
2. Ultralytics YOLOv8 documentation — https://docs.ultralytics.com
3. Bleak — Bluetooth Low Energy platform-agnostic client for Python — https://bleak.readthedocs.io
4. OpenCV documentation, `getPerspectiveTransform` and `perspectiveTransform` — https://docs.opencv.org